# TUTORIAL: Digital Out

Name: HWANG MooYeol ID: 22000814

# I. Introduction

In this lab, we will learn how to control the Digital Output of GPIOs of the MCU board to turn on/off an LED.

The objectives of this lab are to learn how to

- Read and configure registers of digital GPIO of MCU
- Program firmware to control digital input/output pins

### Hardware

NUCLEO -F411RE

### Software

Keil uVision IDE, CMSIS, EC_HAL

### Documentation

STM32 Reference Manual

# II. Basics of GPIO Out

## A. Bit Operation

Fill in the blanks. You should write the answer in both hexa-decimal and binary number.

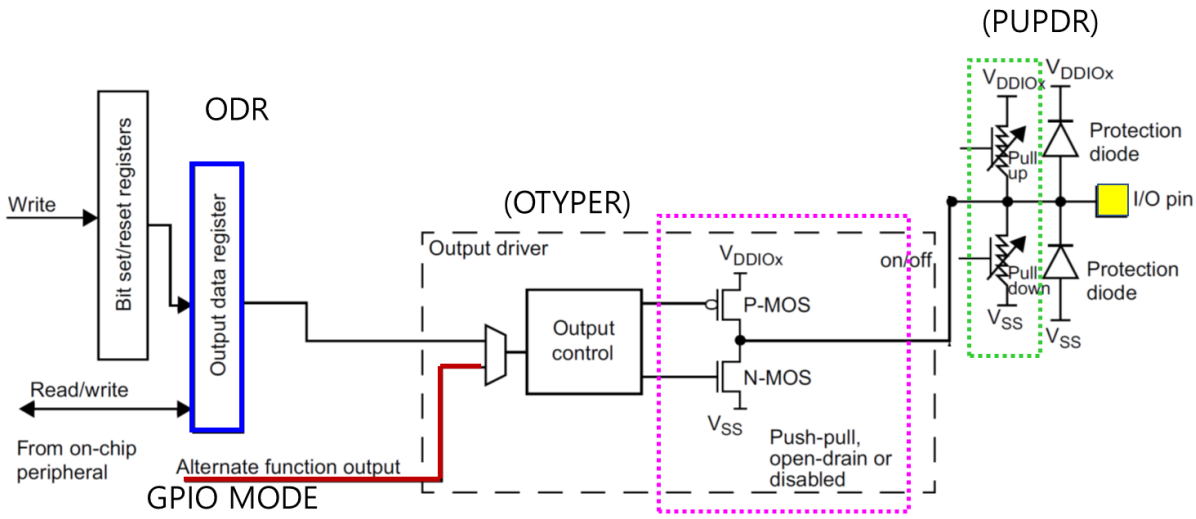
| Bit operation | Description | Answer |
|---|---|---|
| 0xA & 0x2 | AND | 0x2, 0b0010 |
| 0x3 \| 0xC | OR | 0xF, 0b1111 |
| ~0x7 | NOT | 0xF8, 0b1111 1000 |
| 0xF ^ 0x5 | XOR(Toggle) | 0xA, 0b1010 |
| 0x1 << 3 | Shift left | 0x8, 0b1000 |
| 0xC >> 2 | Shift right | 0x3, 0b0011 |
| 0x11 \|= 1<<3 | Set bit | 0x19, 0b0001 1001 |
| 0xFF &= ~1<<4 | Clear bit | 0xEF, 0b1110 1111 |
| val=0x0F &1<<3 | Read bit | 0x8, 0b1000 |

# B. GPIO Digital Out Register

List GPIO registers for this LAB

| Type | Register Name | Description |
|---|---|---|
| GPIO | GPIOx_MODER | Mode: Output/Input/Analog |
| | GPIOx_OTYPER | Output Type: Opendrain/Push-Pull |
| | GPIOx_OSPEEDR | Output Speed: |
| | GPIOx_PUPDR | Pull-Up Pull-Down: |
| | GPIOx_ODR | Output Data Register |

Schematic




Process of GPIOx register initiation

0. Enable Peripheral Clock (**AHB1ENR**)
1. Configure as Digital Output (**MODER**)
2. Configure pull-up/down resistors (**PUPDR**)
3. For Output: Configure Output Type (**OTYPE**)
4. For Output: Configure Output Speed (**OSPEEDR**)
5. Output Data **(ODR)**

# III. Tutorial

## A. Register Configuration

### 1. GPIO: Digital Out - Pin Initialization & Set LED

Port A Pin 5 / Output / Push-Pull / No Pull-Up & No Pull-Down / Default Speed

use **#define LED_PIN 5**

- **MODER:** Output (MODER5[1:0]=01)

```
GPIOA->MODER & = ~(3<<( LED_PIN *2));      // clear bits at both   [10] and [11]
GPIOA->MODER | = 1<<(LED_PIN *2);          // set bit at [10]
```

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| MODER15[1:0] | | MODER14[1:0] | | MODER13[1:0] | | MODER12[1:0] | | MODER11[1:0] | | MODER10[1:0] | | MODER9[1:0] | | MODER8[1:0] | |
| rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| MODER7[1:0] | | MODER6[1:0] | | MODER5[1:0] | | MODER4[1:0] | | MODER3[1:0] | | MODER2[1:0] | | MODER1[1:0] | | MODER0[1:0] | |
| rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 2y:2y+1 **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O direction mode.
00: Input (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

| Register | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Initial | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x |
| Logic | Bitwise OR | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| mask | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Value | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | 0 | 1 | x | x | x | x | x | x | x | x | x | x |

- **OTYPER:** Push-Pull (OT5=0)

```
GPIOA->OTYPER &= ~(1<< LED_PIN)
```

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | | | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| OT15 | OT14 | OT13 | OT12 | OT11 | OT10 | OT9 | OT8 | OT7 | OT6 | OT5 | OT4 | OT3 | OT2 | OT1 | OT0 |
| rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **OTy**: Port x configuration bits (y = 0..15)
These bits are written by software to configure the output type of the I/O port.
0: Output push-pull (reset state)
1: Output open-drain

| Register | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Initial | | | | | | | | | | | | | | | | | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x |
| Logic | Bitwise AND | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| mask | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| Value | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | 0 | x | x | x | x | x |

- **PUPDR:** no pull-up no pull-down (PUPTR5[1:0] = 00)

```
GPIOA->PUPDR &=   ~(3<<( LED_PIN *2)); // Clear
GPIOA->PUPDR |=    0<<( LED_PIN *2);
```

*Paste Register map from reference manual*

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| PUPDR15[1:0] | | PUPDR14[1:0] | | PUPDR13[1:0] | | PUPDR12[1:0] | | PUPDR11[1:0] | | PUPDR10[1:0] | | PUPDR9[1:0] | | PUPDR8[1:0] | |
| rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| PUPDR7[1:0] | | PUPDR6[1:0] | | PUPDR5[1:0] | | PUPDR4[1:0] | | PUPDR3[1:0] | | PUPDR2[1:0] | | PUPDR1[1:0] | | PUPDR0[1:0] | |
| rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 2y:2y+1 **PUPDRy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O pull-up or pull-down
00: No pull-up, pull-down
01: Pull-up
10: Pull-down
11: Reserved

| Register | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Initial** | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x |
| **Logic** | Bitwise OR | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| **mask** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **Value** | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | 0 | 0 | x | x | x | x | x | x | x | x | x | x |

- **ODR:** Set LED (ODR5 = 1)

```
GPIOA->ODR    |= 1<< LED_PIN ;
```

*Paste Register map from reference manual*

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | | | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ODR15 | ODR14 | ODR13 | ODR12 | ODR11 | ODR10 | ODR9 | ODR8 | ODR7 | ODR6 | ODR5 | ODR4 | ODR3 | ODR2 | ODR1 | ODR0 |
| rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data (y = 0..15)

These bits can be read and written by software.

*Note: For atomic bit set/reset, the ODR bits can be individually set and reset by writing to the GPIOx_BSRR register (x = A..E and H).*

| Register | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Initial** | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x |
| **Logic** | Bitwise AND | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| **mask** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| **Value** | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | x | 1 | x | x | x | x | x |

# B. Programming

**Preparation**

- Open the program ‘Keil uVision5’ and create a new project.
- “ /repos/EC/Tutorial/TU_GPIO_Digital_Out_LED/”
- Name the project as ‘**TU_GPIO_Digital_Out_LED**’.
- Create a new item(file) and name it as ‘**TU_GPIO_Digital_Out_LED.c’**
- Copy and paste from the source code ‘**TU_GPIO_Digital_Out_LED_student.c**’.
- Download ecRCC_student.h and ecRCC_student.c. Then, change the file names as **ecRCC2.h** and **ecRCC2.c**.
- Save them in \lib folder: “**repos/EC/include**”

**Exercise**

This is an example code of turning ON LED *without* button input. Do not worry if you do not understand what this code means yet. You will learn one by one in the following few weeks.

Fill in the empty spaces in the code. Then, compile(F7) and flash(F8) the source code onto t he MCU board. Verify the program by checking if the LED is turned ON.

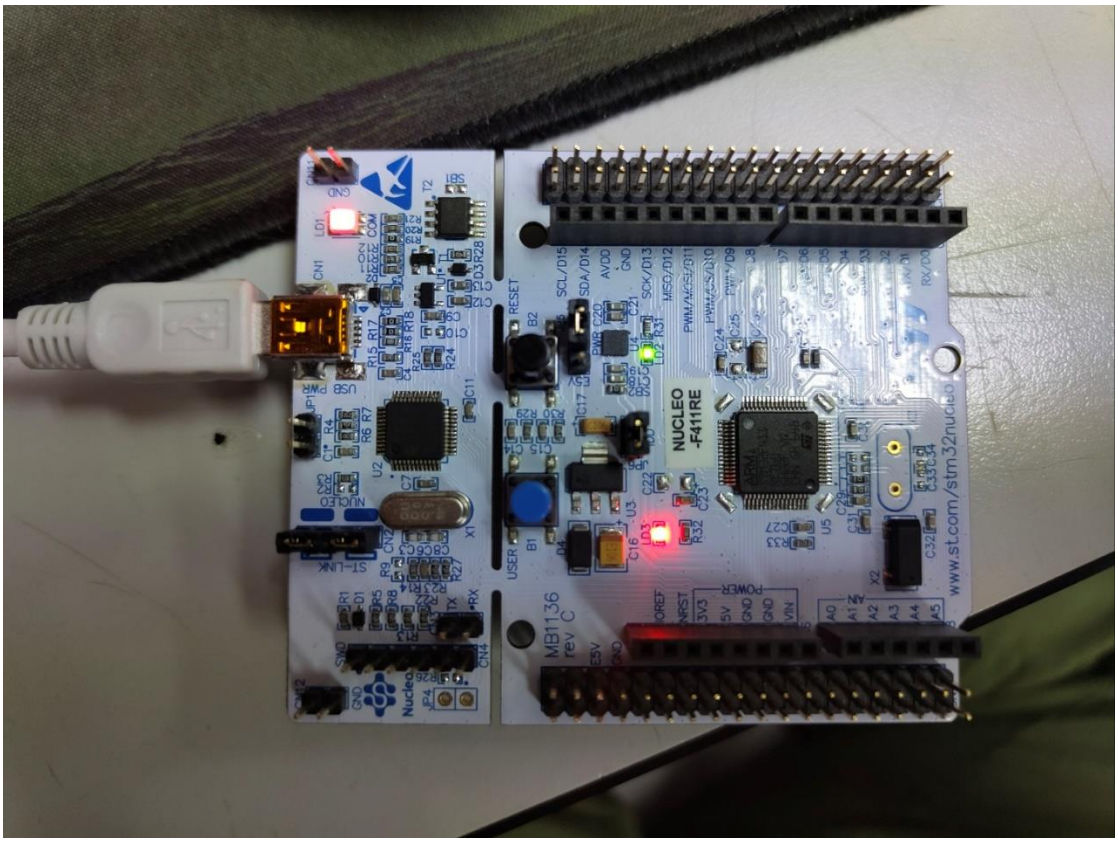
Embedded Controller

## My CODE

TU_GPIO_Digital_Out_LED.c

```c
/**
  ******************************************************************************
  * @author  SSSLAB
  * @Mod     2024-9-9 by HWANG MooYeol
  * @brief   Embedded Controller:  Tutorial Digital Out
  *          - Turn on LED LD2
  *
  ******************************************************************************
*/


// GPIO Mode        : Input(00), Output(01), AlterFunc(10), Analog(11, reset)
// GPIO Speed       : Low speed (00), Medium speed (01), Fast speed (10), High speed (11)
// GPIO Output Type: Output push-pull (0, reset), Output open drain (1)
// GPIO Push-Pull  : No pull-up, pull-down (00), Pull-up (01), Pull-down (10), Reserved (11)


#include "stm32f4xx.h"
#include "ecRCC2.h"

#define LED_PIN    5    //LD2


int main(void) {
    /* Part 1. RCC GPIOA Register Setting */
    RCC_HSI_init();
    RCC_GPIOA_enable();

    /* Part 2. GPIO Register Setting */
    // GPIO Mode Register
    GPIOA->MODER &= ~(3<<(LED_PIN*2));        // Clear '00' for Pin 5
    GPIOA->MODER |=   1<<(LED_PIN*2);         // Set '01' for Pin 5

    // GPIO Output Type Register
    GPIOA->OTYPER &= ~(1<<(LED_PIN*2));       // Clear '00'
    GPIOA->OTYPER |=   0<<(LED_PIN*2);        // 0:Push-Pull

    // GPIO Pull-Up/Pull-Down Register
    GPIOA->PUPDR  &= ~(3<<(LED_PIN*2));       // 00: none

    // GPIO Output Speed Register
    GPIOA->OSPEEDR &= ~(3<<(LED_PIN*2));
    GPIOA->OSPEEDR |= 2<<(LED_PIN*2);           //10:Fast Speed

    // Dead loop & program hangs here
    while(1){
      //   GPIOA->ODR = 1UL << LED_PIN;   // Set LED_PIN = H, others=L
      GPIOA->ODR |= (1UL << LED_PIN);     // Change only LED_PIN = H
    }
}
```

## MCU board Result



# Appendix

[See here for MCU resources]

1. Pin Configuration of NUCLE-F401RE

**Figure 18. NUCLEO-F401RE**

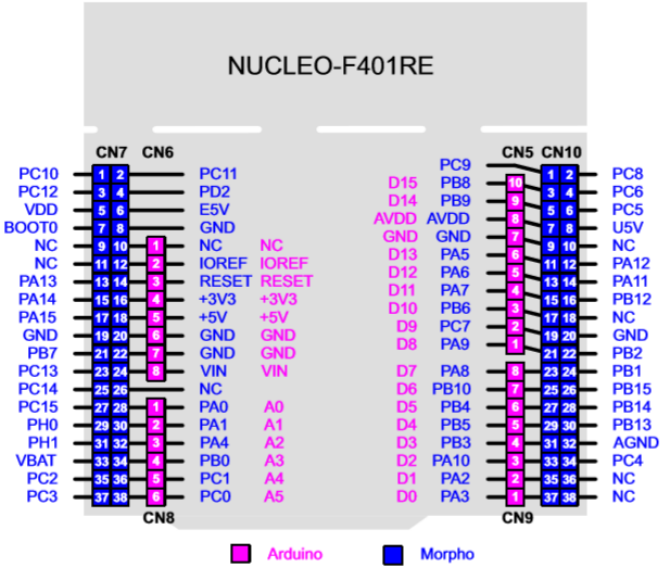
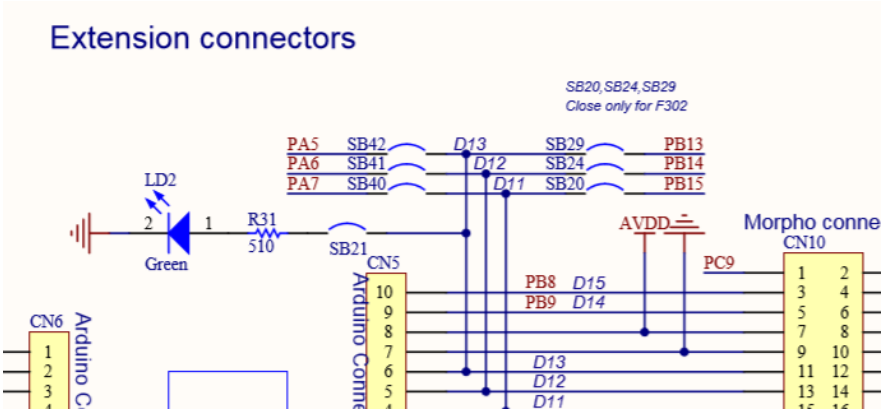
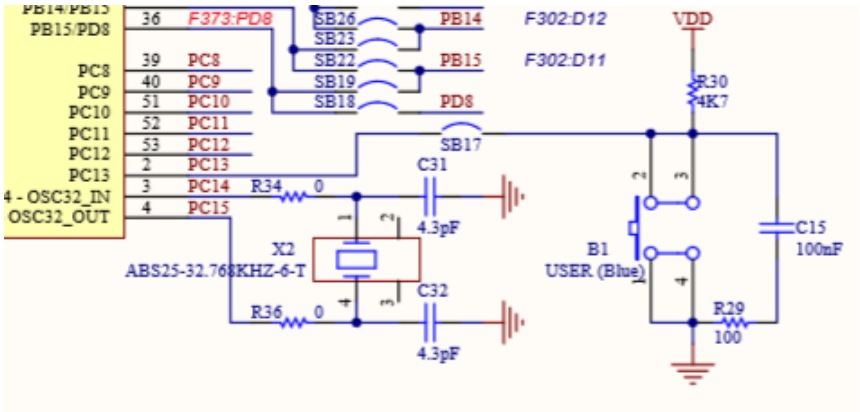



**Table 29. ST morpho connector on NUCLEO-F401RE, NUCLEO-F411RE, NUCLEO-F446RE**

| CN7 odd pins | | CN7 even pins | | CN10 odd pins | | CN10 even pins | |
|---|---|---|---|---|---|---|---|
| Pin | Name | Name | Pin | Pin | Name | Name | Pin |
| 1 | PC10 | PC11 | 2 | 1 | PC9 | PC8 | 2 |
| 3 | PC12 | PD2 | 4 | 3 | PB8 | PC6 | 4 |
| 5 | VDD | E5V | 6 | 5 | PB9 | PC5 | 6 |
| 7 | BOOT0(1) | GND | 8 | 7 | AVDD | U5V(2) | 8 |
| 9 | - | - | 10 | 9 | GND | - | 10 |
| 11 | - | IOREF | 12 | 11 | PA5 | PA12 | 12 |
| 13 | PA13(3) | RESET | 14 | 13 | PA6 | PA11 | 14 |
| 15 | PA14(3) | +3.3V | 16 | 15 | PA7 | PB12 | 16 |
| 17 | PA15 | +5V | 18 | 17 | PB6 | - | 18 |
| 19 | GND | GND | 20 | 19 | PC7 | GND | 20 |
| 21 | PB7 | GND | 22 | 21 | PA9 | PB2 | 22 |
| 23 | PC13 | VIN | 24 | 23 | PA8 | PB1 | 24 |
| 25 | PC14 | - | 26 | 25 | PB10 | PB15 | 26 |
| 27 | PC15 | PA0 | 28 | 27 | PB4 | PB14 | 28 |
| 29 | PH0 | PA1 | 30 | 29 | PB5 | PB13 | 30 |
| 31 | PH1 | PA4 | 32 | 31 | PB3 | AGND | 32 |
| 33 | VBAT | PB0 | 34 | 33 | PA10 | PC4 | 34 |
| 35 | PC2 | PC1 or PB9(4) | 36 | 35 | PA2 | - | 36 |
| 37 | PC3 | PC0 or PB8(4) | 38 | 37 | PA3 | - | 38 |

1. Default state of BOOT0 is 0. It can be set to 1 when a jumper is on pin5-7 of CN7.Two unused jumpers are available on CN11 and CN12 (bottom side of the board).
2. U5V is 5 V power from ST-LINK/V2-1 USB connector and it rises before +5V.
3. PA13 and PA14 share with SWD signals connected to ST-LINK/V2-1, it is not recommend to use them as IO pins if ST-LINK part is not cut.
4. Refer to *Table 10: Solder bridges* for details.

2. LED/Button Circuit Diagram